

Mid term project Report

Project name: Code Energy Profiler.

Course: CSE407 (Green Computing)

Section: 02

Submitted To:

Ahmed Abdal Shafi Rasel

Lecturer

Department of Computer Science & Engineering

Submitted By:

Name: Md Ferdows

Id no: 2021-3-60-102

Submission Date: 07.11.2025

Topic Name: Code Energy Profiler of Searching Algorithms (Binary , Liner Search) of Python Language.

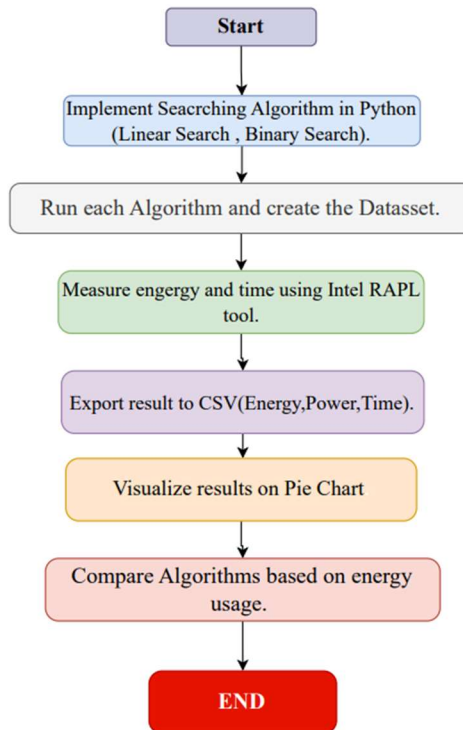
Executive Summary:

This project investigates the energy efficiency and performance of two fundamental searching algorithms — **Linear Search** and **Binary Search** — implemented using Python. The main objective is to evaluate and compare these algorithms based on their energy consumption, power usage, and execution time. With the growing focus on green computing and energy-efficient software development, this study aims to highlight how algorithm design influences energy utilization during program execution. The process begins by implementing both search algorithms and running them on datasets of varying sizes to simulate real-world scenarios. The **Intel RAPL (Running Average Power Limit)** tool is used to accurately measure the energy consumption and execution time for each algorithm during runtime. The collected data, including parameters such as energy (Joules), power (Watts), and time (seconds), is then exported to a CSV file for structured analysis. Subsequently, Microsoft Excel is utilized to visualize the collected results. A pie chart is created to represent the proportion of energy usage and power consumption by each algorithm. This visualization makes it easier to understand the differences between the two approaches in terms of energy efficiency. After analyzing the visual data, the project concludes by comparing both algorithms. The comparison reveals that Binary Search, being more efficient in terms of algorithmic complexity (**$O(\log n)$**), consumes significantly less energy and time compared to Linear Search, which has a time complexity of **$O(n)$** . Overall, the experiment demonstrates that algorithm efficiency not only affects execution speed but also plays a crucial role in reducing energy consumption. This study emphasizes the importance of designing and choosing algorithms that are both computationally and energy efficient, contributing to sustainable computing practices and improved system performance.

Brief Description of the work:

The paper analyzes and compares Python-coded algorithms for searching with respect to their energy efficiency. In this regard, two common algorithms, namely **Linear Search** and **Binary Search**, are implemented for the purpose of investigating these using performance metrics such as energy, power, and execution time. The algorithms were run numerous times in order to provide datasets that can be used for analysis. For the measurement of energy and time, **Intel RAPL (Running Average Power Limit)** was utilized. Energy, power, and time data were collected and then exported in a CSV file for better organization and analysis. Subsequently, Microsoft Excel was utilized to visualize the results through pie charts, which could represent the proportion of consumption from each algorithm. The algorithms were then compared with respect to energy efficiency. The studies clearly proved that **Binary Search** consumes **less energy** and is **faster** than **Linear Search**. It serves to emphasize how algorithm design influences energy use efficiency.

Methodology(flowchart/diagram):



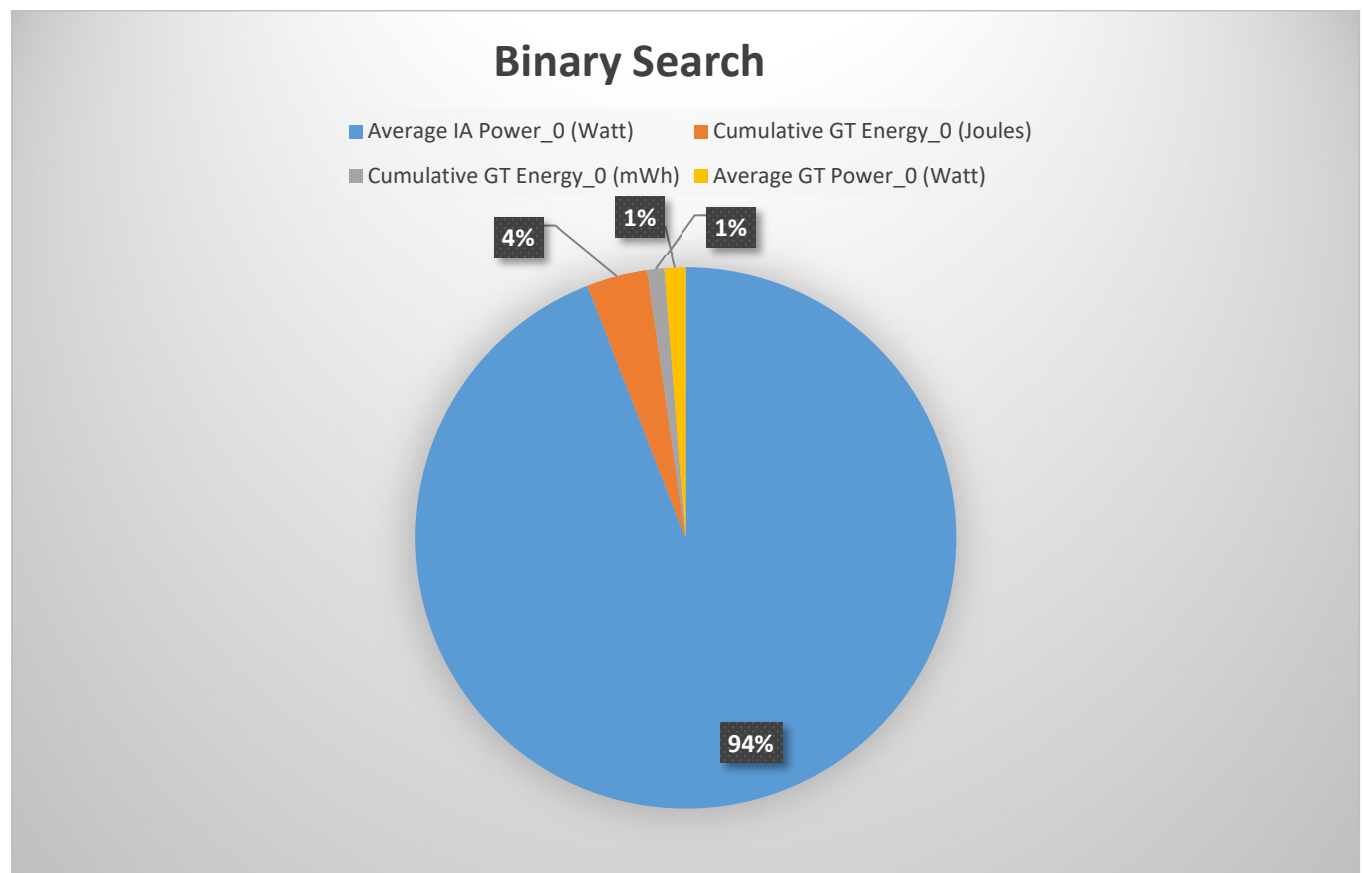
Detailed description of steps:

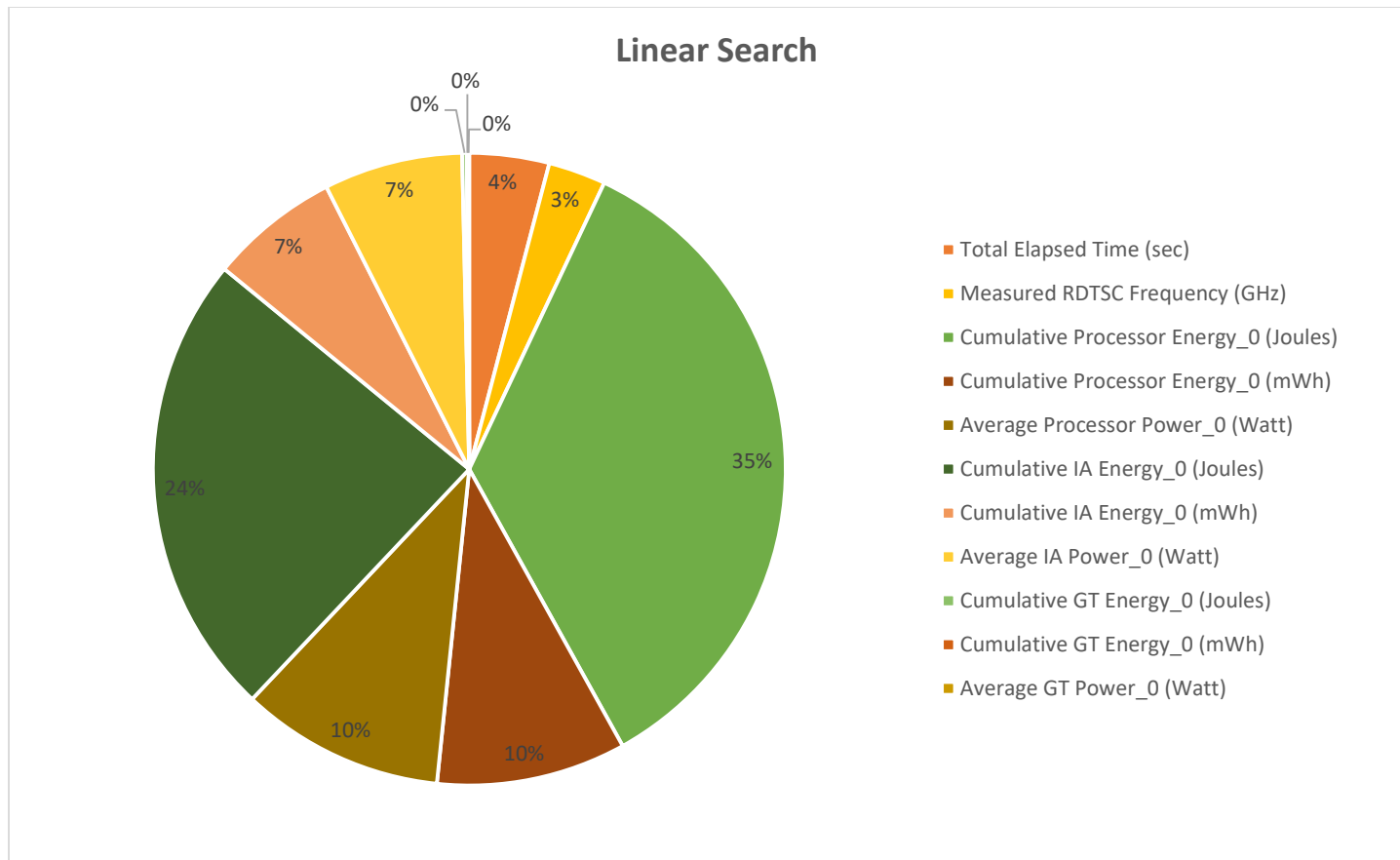
This project deals with the analysis of the energy efficiency of various searching algorithms using Python. The project starts with the implementation of **two search algorithms: Linear Search and Binary Search**. Each of them is executed to create a dataset for analysis. Then, energy and execution time are measured for each algorithm using **the Intel RAPL tool**. The recorded data, which includes energy, power, and time, is then exported in the form of a CSV file to get better insights into the data. With the help of Microsoft Excel, the results are visualized in the form of a pie to clearly show the usage distribution of energy among the algorithms. Finally, the algorithms are compared in respect of their energy efficiency to point out which algorithm performs better regarding power consumption. By this experiment, one can clearly demonstrate the relationship between algorithm design and energy efficiency in computing.

Challenges and hiccups:

During the project, several challenges were encountered while implementing and analyzing the algorithms. One major difficulty was accurately measuring energy consumption using the **Intel RAPL tool**, as it required proper setup, calibration, and understanding of how the readings correspond to the CPU's energy usage. Another challenge was ensuring consistent testing conditions, since small background processes or system load variations could affect the results. Creating and handling large datasets to test the algorithms also caused performance delays and memory issues at times. Additionally, exporting and formatting the measured data into CSV files required careful handling to avoid data loss or incorrect readings. Visualizing the results using **Microsoft Excel** also posed minor difficulties, particularly in creating clear and informative pie charts. Despite these hiccups, all challenges were successfully managed through troubleshooting, repeated testing, and careful validation, leading to reliable and meaningful results for the project.

Demonstration:





It can be seen from the analysis of the two charts that **Binary Search** is much more power-efficient compared to Linear Search. In the chart for **Binary Search**, most of the power consumed comes from the average IA power, about **94%**, while others only take **1–4%**, which shows that minimal resources are used. On the other hand, in the **Linear Search** chart, there is distributed and higher energy usage across several parameters; the main contribution to this comes from processor and IA energy. This means Linear Search requires more power and time because it proceeds linearly. Overall, **Binary Search** is much better since it is **faster** and consumes **less energy**.

Discussion of the issue:

(a) Checklist:

A checklist is a straightforward yet powerful device for ensuring that all necessary steps or tasks are completed correctly and in the proper order. In research or experimental analysis, a checklist ensures that such work is consistent, accurate, and well-organized. It enables researchers to systematically double-check data collection, analysis, and reporting procedures. Using another example, when comparing Binary and Linear Search for energy efficiency, a checklist would ensure that all possible measurements, such as power consumption, execution time, and resource utilization, are appropriately measured and compared.

(b) Discussion on Issues:

Throughout the experiment, several problems cropped up. A major one was the inconsistency in data as the system fluctuated during measurement, which had some slight impact on the accuracy of the power reading. Another issue was the variation in CPU utilization, as background processes sometimes influenced energy results. While plotting the data using Excel through pie charts, maintaining uniform scaling and correct labeling was hard to achieve. It was also critical that the running conditions were the same for both algorithms in order to make fair comparisons. Despite these various challenges, the use of checklists and repeated testing minimized errors. In general, the discussion here emphasizes the importance of careful experimental control and verification using checklists for realizing valid and reliable results in algorithm performance analysis.

Appendices:

Code:

1. [Binary Search.](#)
2. [Linear Search.](#)
3. [Dataset.](#)